



Linux Fundamentals

0%



Section 23 / 30

Containerization

Containerization is the process of packaging and running applications in isolated environments, typically referred to as containers. These containers provide lightweight, consistent environments for applications to run, ensuring that they behave the same way, regardless of where they are deployed. Technologies like Docker, Docker Compose, and Linux Containers (LXC) make containerization possible, primarily in Linux-based systems. Containers differ from virtual machines in that they share the host system's kernel, making them far more lightweight and efficient. With these technologies, users can quickly create, deploy, and manage applications with improved security, portability, and scalability.

Containers are highly configurable, allowing users to tailor them to their specific needs, and their lightweight nature makes it easy to run multiple containers simultaneously on the same host system. This feature is particularly advantageous for scaling applications and managing complex microservice architectures.

Imagine you're organizing a big concert, and each band needs their own customized stage setup. Instead of building a completely new stage for each band (which would be like using a virtual machine), you create portable, self-contained "stage pods" that include everything the band needs

lights, instruments, speakers, etc. These pods are lightweight, reusable, and can be easily moved from venue to venue. The key is that the pods all work seamlessly on the same main stage (the host system), but each one is isolated enough that no band's setup interferes with the others.

In the same way, containers package an application with all its required tools and settings, allowing it to run consistently across different systems without conflict, all while sharing the same "main stage" (the underlying system's resources).

Security is a critical aspect of containerization. Containers isolate applications from the host system and from each other, providing a barrier that reduces the risk of malicious activities affecting the host or other containers. This isolation, along with proper configuration and hardening techniques, adds an additional layer of security. However, it's important to note that containers do not offer the same level of isolation as traditional virtual machines. If not properly managed, vulnerabilities such as privilege escalation or container escapes can be exploited to gain unauthorized access to the host system or other containers.

In addition to enhanced security and resource efficiency, containers make applications easier to deploy, manage, and scale. Since containers encapsulate everything the application needs (e.g., libraries, dependencies), they allow for consistency across development, testing, and production environments. This portability ensures that applications run reliably across different environments.

However, it is important to recognize that, despite their advantages, containers are not immune to security risks. There are methods that we can use to escalate privileges or escape the isolation that containers provide.

Dockers

Docker is an open-source platform for automating the deployment of applications as self-contained units called containers. It uses a layered filesystem and resource isolation features to provide flexibility and portability. Additionally, it provides a robust set of tools for creating, deploying, and managing applications, which helps streamline the containerization process.

Imagine Docker containers as a sealed lunchbox. You can eat the food (run applications) inside, but once you close the box (stop the container), everything resets. To make a new lunchbox (new container) with updated contents (modified configurations), you create a new recipe (Dockerfile) based on the original. When serving multiple lunchboxes in a restaurant (production), you'd use a kitchen system (Kubernetes/Docker Compose) to manage all the orders smoothly.

Install Docker-Engine

Installing Docker is relatively straightforward. We can use the following script to install it on a Ubuntu host:

```
#!/bin/bash

# Preparation
sudo apt update -y
sudo apt install ca-certificates curl gnupg lsb-release -y
sudo mkdir -m 0755 -p /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee
```

bash

```
/etc/apt/sources.list.d/docker.list > /dev/null

# Install Docker Engine
sudo apt update -y
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-
compose-plugin -y

# Add user htb-student to the Docker group
sudo usermod -aG docker htb-student
echo '[!] You need to log out and log back in for the group changes to take effect.'

# Test Docker installation
docker run hello-world
```

The Docker engine and specific Docker images are needed to run a container. These can be obtained from the **Docker Hub**, a repository of pre-made images, or created by the user. The Docker Hub is a cloud-based registry for software repositories or a library for Docker images. It is divided into a **public** and a **private** area. The public area allows users to upload and share images with the community. It also contains official images from the Docker development team and established open-source projects. Images uploaded to a private area of the registry are not publicly accessible. They can be shared within a company or with teams and acquaintances.

Creating a Docker image is done by creating a **Dockerfile**, which contains all the instructions the Docker engine needs to create the container. We can use Docker containers as our “file hosting” server when transferring specific files to our target systems. Therefore, we must create a **Dockerfile** based on Ubuntu 22.04 with **Apache** and **SSH** server running. With this, we can use **scp** to transfer files to the docker image, and Apache allows us to host files and use tools

like `curl`, `wget`, and others on the target system to download the required files. Such a `Dockerfile` could look like the following:

Dockerfile

```
bash
# Use the latest Ubuntu 22.04 LTS as the base image
FROM ubuntu:22.04

# Update the package repository and install the required packages
RUN apt-get update && \
    apt-get install -y \
        apache2 \
        openssh-server \
        && \
    rm -rf /var/lib/apt/lists/*

# Create a new user called "docker-user"
RUN useradd -m docker-user && \
    echo "docker-user:password" | chpasswd

# Give the docker-user user full access to the Apache and SSH services
RUN chown -R docker-user:docker-user /var/www/html && \
    chown -R docker-user:docker-user /var/run/apache2 && \
    chown -R docker-user:docker-user /var/log/apache2 && \
    chown -R docker-user:docker-user /var/lock/apache2 && \
    usermod -aG sudo docker-user && \
    echo "docker-user ALL=(ALL) NOPASSWD: ALL" >> /etc/sudoers

# Expose the required ports
```

```
EXPOSE 22 80
```

```
# Start the SSH and Apache services
```

```
CMD service ssh start && /usr/sbin/apache2ctl -D FOREGROUND
```

After we have defined our Dockerfile, we need to convert it into an image. With the `build` command, we take the directory with the Dockerfile, execute the steps from the `Dockerfile`, and store the image in our local Docker Engine. If one of the steps fails due to an error, the container creation will be aborted. With the option `-t`, we give our container a tag, so it is easier to identify and work with later.

Docker Build

```
athane@htb[/htb]$ docker build -t FS_docker .
```

shellsession

Once the Docker image has been created, it can be executed through the Docker engine, making it a very efficient and easy way to run a container. It is similar to the virtual machine concept, based on images. Still, these images are read-only templates and provide the file system necessary for runtime and all parameters. A container can be considered a running process of an image. When a container is to be started on a system, a package with the respective image is first loaded if unavailable locally. We can start the container by the following command `docker run`:

Docker Run - Syntax

```
athane@htb[/htb]$ docker run -p <host port>:<docker port> -d <docker container name>
```

shellsession

Docker Run

```
athane@htb[/htb]$ docker run -p 8022:22 -p 8080:80 -d FS_docker
```

shellsession

In this case, we start a new container from the image `FS_docker` and map the host ports 8022 and 8080 to container ports 22 and 80, respectively. The container runs in the background, allowing us to access the SSH and HTTP services inside the container using the specified host ports.

Docker Management

When managing Docker containers, Docker provides a comprehensive suite of tools that enable us to easily create, deploy, and manage containers. With these powerful tools, we can list, start and stop containers and effectively manage them, ensuring seamless execution of applications. Some of the most commonly used Docker management commands are:

Command	Description
<code>docker ps</code>	List all running containers
<code>docker stop</code>	Stop a running container.
<code>docker start</code>	Start a stopped container.

Command	Description
<code>docker restart</code>	Restart a running container.
<code>docker rm</code>	Remove a container.
<code>docker rmi</code>	Remove a Docker image.
<code>docker logs</code>	View the logs of a container.

It's important to note that Docker commands can be combined with various options to add extra functionality. For example, you can specify which ports to expose, mount volumes to retain data, or set environment variables to configure your containers. This flexibility allows you to customize your Docker containers to meet specific needs and requirements.

When working with Docker images, it's crucial to understand that any changes made to a running container based on an image are not automatically saved to the image. To preserve these changes, you need to create a new image that includes them. This is done by writing a new Dockerfile, which starts with the `FROM` statement (specifying the base image) and then includes the necessary commands to apply the changes. Once the Dockerfile is ready, you can use the `docker build` command to build the new image and assign it a unique tag to identify it. This process ensures that the original image remains unchanged, while the new image reflects the updates.

It's also important to remember that Docker containers are stateless by design, meaning that any changes made inside a running container (e.g., modifying files) are lost once the container is

stopped or removed. For this reason, it's best practice to use volumes to persist data outside of the container or store application state.

In production environments, managing containers at scale becomes more complex. Tools like Docker Compose or Kubernetes help orchestrate containers, enabling you to manage, scale, and link multiple containers efficiently.

Linux Containers

Linux Containers (**LXC**) is a lightweight virtualization technology that allows multiple isolated Linux systems (called containers) to run on a single host. LXC uses key resource isolation features, such as control groups (**cgroups**) and **namespaces** , to ensure that each container operates independently. Unlike traditional virtual machines, which require a full OS for each instance, containers share the host's kernel, making LXC more efficient in terms of resource usage.

LXC provides a comprehensive set of tools and APIs for managing and configuring containers, making it a popular choice for containerization on Linux systems. However, while LXC and Docker are both containerization technologies, they serve different purposes and have unique features.

Docker builds upon the idea of containerization by adding ease of use and portability, which has made it highly popular in the world of DevOps. Docker emphasizes packaging applications with all their dependencies in a portable "image", allowing them to be easily deployed across different environments. However, there are some differences between the two that can be distinguished based on the following categories:

Category	Description
Approach	LXC is often seen as a more traditional, system-level containerization tool, focusing on creating isolated Linux environments that behave like lightweight virtual machines. Docker, on the other hand, is application-focused, meaning it is optimized for packaging and deploying single applications or microservices.
Image building	Docker uses a standardized image format (Docker images) that includes everything needed to run an application (code, libraries, configurations). LXC, while capable of similar functionality, typically requires more manual setup for building and managing environments.
Portability	Docker excels in portability. Its container images can be easily shared across different systems via Docker Hub or other registries. LXC environments are less portable in this sense, as they are more tightly integrated with the host system's configuration.
Easy of use	Docker is designed with simplicity in mind, offering a user-friendly CLI and extensive community support. LXC, while powerful, may require more in-depth knowledge of Linux system administration, making it less straightforward for beginners.
Security	Docker containers are generally more secure out of the box, thanks to additional isolation layers like AppArmor and SELinux, along with its read-only filesystem feature. LXC containers, while secure, may need additional configurations to match the level of isolation Docker offers by default. Interestingly enough, when misconfigured, both Docker and LXC can present a vector for local privilege escalation (these techniques are covered in depth in our Linux Local Privilege Escalation module).

In LXC, images are manually built by creating a root filesystem and installing the necessary packages and configurations. Those containers are tied to the host system, may not be easily portable, and may require more technical expertise to configure and manage.

On the other hand, Docker is an application-centric platform that builds on top of LXC and provides a more user-friendly interface for containerization. Its images are built using a Dockerfile, which specifies the base image and the steps required to build the image. Those images are designed to be portable so they can be easily moved from one environment to another.

To install LXC on a Linux distribution, we can use the distribution's package manager. For example, on Ubuntu, we can use the `apt` package manager to install LXC with the following command:

Install LXC

```
athane@htb[/htb]$ sudo apt install lxc -y
```

shellsession

Once LXC is installed, we can start creating and managing containers on the Linux host. It is worth noting that LXC requires the Linux kernel to support the necessary features for containerization. Most modern Linux kernels have built-in support for containerization, but some older kernels may require additional configuration or patching to enable support for LXC.

Creating an LXC Container

To create a new LXC container, we can use the `lxc-create` command followed by the container's name and the template to use. For example, to create a new Ubuntu container named

`linuxcontainer`, we can use the following command:

```
athane@htb[/htb]$ sudo lxc-create -n linuxcontainer -t ubuntu
```

shellsession

Managing LXC Containers

When working with LXC containers, several tasks are involved in managing them. These tasks include creating new containers, configuring their settings, starting and stopping them as necessary, and monitoring their performance. Fortunately, there are many command-line tools and configuration files available that can assist with these tasks. These tools enable us to quickly and easily manage our containers, ensuring they are optimized for our specific needs and requirements. By leveraging these tools effectively, we can ensure that our LXC containers run efficiently and effectively, allowing us to maximize our system's performance and capabilities.

Command	Description
<code>lxc-ls</code>	List all existing containers
<code>lxc-stop -n <container></code>	Stop a running container.
<code>lxc-start -n <container></code>	Start a stopped container.
<code>lxc-restart -n <container></code>	Restart a running container.

Command	Description
<code>lxc-config -n <container name> -s storage</code>	Manage container storage
<code>lxc-config -n <container name> -s network</code>	Manage container network settings
<code>lxc-config -n <container name> -s security</code>	Manage container security settings
<code>lxc-attach -n <container></code>	Connect to a container.
<code>lxc-attach -n <container> -f /path/to/share</code>	Connect to a container and share a specific directory or file.

As penetration testers, we often face situations where we need to test software or systems that have complex dependencies or configurations that are difficult to replicate on our local machines. This is where Linux containers become extremely useful. A Linux container is a lightweight, standalone package that includes everything needed to run a specific piece of software such as the code, libraries, and configuration files. Containers offer an isolated environment, allowing the software to run consistently across any Linux machine, regardless of the host system's configuration.

Containers are particularly useful because they allow us to quickly create and run isolated environments tailored to our specific testing needs. For instance, if we need to test a web application that depends on a particular version of a database or web server, we can create a

container with the exact versions and configurations we need. This eliminates the hassle of manually setting up these components on our machine, which can be time-consuming and prone to errors.

We can also use them to test exploits or malware in a controlled environment where we create a container that simulates a vulnerable system or network and then use that container to safely test exploits without risking damaging our machines or networks. However, it is important to configure LXC container security to prevent unauthorized access or malicious activities inside the container. This can be achieved by implementing several security measures, such as:

- Restricting access to the container
- Limiting resources
- Isolating the container from the host
- Enforcing mandatory access control
- Keeping the container up to date

LXC containers can be accessed using various methods, such as SSH or console. It is recommended to restrict access to the container by disabling unnecessary services, using secure protocols, and enforcing strong authentication mechanisms. For example, we can disable SSH access to the container by removing the `openssh-server` package or by configuring SSH only to allow access from trusted IP addresses. Those containers also share the same kernel as the host system, meaning they can access all the resources available on the system. We can use resource limits or quotas to prevent containers from consuming excessive resources. For example, we can use `cgroups` to limit the amount of CPU, memory, or disk space that a container can use.

Securing LXC

Let us limit the resources to the container. In order to configure `cgroups` for LXC and limit the CPU and memory, a container can create a new configuration file in the `/usr/share/lxc/config/<container name>.conf` directory with the name of our container. For example, to create a configuration file for a container named `linuxcontainer`, we can use the following command:

```
athane@htb[/htb]$ sudo vim /usr/share/lxc/config/linuxcontainer.conf
```

shellsession

In this configuration file, we can add the following lines to limit the CPU and memory the container can use.

```
lxc.cgroup.cpu.shares = 512  
lxc.cgroup.memory.limit_in_bytes = 512M
```

txt

When working with containers, it is important to understand the `lxc.cgroup.cpu.shares` parameter. This parameter determines the CPU time a container can use in relation to the other containers on the system. By default, this value is set to 1024, meaning the container can use up to its fair share of CPU time. However, if we set this value to 512, for example, the container can only use half of the CPU time available on the system. This can be a useful way to manage resources and ensure all containers have the necessary access to CPU time.

One of the key parameters in controlling the resource allocation of a container is the `lxc.cgroup.memory.limit_in_bytes` parameter. This parameter allows you to set the maximum amount of memory a container can use. It's important to note that this value can be

specified in a variety of units, including bytes, kilobytes (K), megabytes (M), gigabytes (G), or terabytes (T), allowing for a high degree of granularity in defining container resource limits. After adding these two lines, we can save and close the file by typing:

- [Esc]
- :
- wq

To apply these changes, we must restart the LXC service.

```
athane@htb[/htb]$ sudo systemctl restart lxc.service
```

shellsession

LXC use **namespaces** to provide an isolated environment for processes, networks, and file systems from the host system. Namespaces are a feature of the Linux kernel that allows for creating isolated environments by providing an abstraction of system resources.

Namespaces are a crucial aspect of containerization as they provide a high degree of isolation for the container's processes, network interfaces, routing tables, and firewall rules. Each container is allocated a unique process ID (**pid**) number space, isolated from the host system's process IDs. This ensures that the container's processes cannot interfere with the host system's processes, enhancing system stability and reliability. Additionally, each container has its own network interfaces (**net**), routing tables, and firewall rules, which are completely separate from the host system's network interfaces. Any network-related activity within the container is cordoned off from the host system's network, providing an extra layer of network security.

Moreover, containers come with their own root file system (`mnt`), which is entirely different from the host system's root file system. This separation between the two ensures that any changes or modifications made within the container's file system do not affect the host system's file system. However, it is important to remember that while namespaces provide a high level of isolation, they do not provide complete security. Therefore, it is always advisable to implement additional security measures to further protect the container and the host system from potential security breaches.

Here are 9 optional exercises to practice LXC:

- 1 Install LXC on your machine and create your first container.
- 2 Configure the network settings for your LXC container.
- 3 Create a custom LXC image and use it to launch a new container.
- 4 Configure resource limits for your LXC containers (CPU, memory, disk space).
- 5 Explore the `lxc-*` commands for managing containers.
- 6 Use LXC to create a container running a specific version of a web server (e.g., Apache, Nginx).
- 7 Configure SSH access to your LXC containers and connect to them remotely.

- 8 Create a container with persistence, so changes made to the container are saved and can be reused.
- 9 Use LXC to test software in a controlled environment, such as a vulnerable web application or malware.

Connect to HTB



23 / 30 Sections



Mark Complete & Next